

1 Bash Shell dan Shell Basic

Bab ini membahas Bash sebagai shell default yang umum digunakan pada Linux, cara kerja file konfigurasi Bash, variabel lingkungan dan PATH, pembuatan alias dan fungsi shell, fitur completion dan history, wildcard dan ekspansi nama file, serta quoting dan escaping. Fokus pembahasan diarahkan pada kebutuhan praktikum dan administrasi sistem sehari-hari: bekerja lebih cepat di terminal, mengurangi kesalahan saat mengetik perintah, dan membangun kebiasaan kerja yang aman.

Contoh: Bash sebagai “panel kendali” administrator

Bayangkan Anda sedang mengelola sebuah server tanpa antarmuka grafis. Tidak ada tombol klik, tidak ada menu visual, dan tidak ada jendela aplikasi. Semua interaksi dilakukan melalui terminal. Dalam kondisi seperti ini, **shell** berperan sebagai panel kendali utama antara pengguna dan sistem operasi. Bash menerima perintah yang Anda ketik, menjalankan program, mengatur variabel lingkungan, menangani redirection, membantu pencarian perintah melalui history, bahkan memungkinkan Anda membuat shortcut kerja sendiri melalui alias dan fungsi. Karena itu, memahami Bash bukan hanya soal “bisa mengetik command”, tetapi juga soal membangun workflow kerja yang lebih cepat, konsisten, dan aman.

1.1 Pengenalan Bash sebagai Shell Default di Linux

Bash (*Bourne Again Shell*) adalah shell yang sangat umum dijumpai pada sistem Linux modern. Shell ini menjadi antarmuka yang menerjemahkan perintah dari pengguna menjadi aksi yang dapat dijalankan sistem operasi. Bash juga menyediakan banyak fasilitas praktis seperti history, completion, variabel lingkungan, alias, fungsi, dan scripting sederhana.

Dalam konteks administrasi sistem, Bash penting karena hampir seluruh aktivitas operasional pada server Linux dilakukan dari terminal: membaca log, mengelola

service, memeriksa disk, membuat backup, hingga menjalankan otomasi sederhana. Jika terminal adalah “ruang kerja” Anda, maka Bash adalah alat kerja utamanya.

Fungsi Perintah yang Digunakan

- `echo $SHELL`: melihat shell login pengguna.
- `echo $0`: melihat nama shell aktif saat ini.
- `ps -p $$ -o pid,ppid,args=`: melihat proses shell yang sedang aktif.
- `bash -version`: melihat versi Bash.
- `help`: bantuan untuk *builtin command*.
- `man bash`: dokumentasi lengkap Bash.
- `mkdir, touch`: menyiapkan workspace praktikum.

Tips

Pada Linux, istilah *default shell* bisa merujuk ke shell login pengguna, sedangkan shell yang sedang aktif pada terminal saat ini bisa saja berbeda. Karena itu, biasanya membedakan `$SHELL` dan `$0`.

Praktikum 6.1 — Mengenali Bash dan Menyiapkan Workspace

Tujuan: mengenali shell aktif dan menyiapkan area kerja yang aman untuk seluruh praktikum bab ini.

1. Lihat shell login dan shell aktif saat ini:

```
echo "Shell login : $SHELL"
echo "Shell aktif : $0"
bash --version | head -n 1
```

Kode 1.1: Melihat shell login dan shell aktif

2. Lihat proses shell yang sedang berjalan:

```
echo $$
ps -p $$ -o pid,ppid,args=
```

Kode 1.2: Melihat proses shell aktif

3. Buat workspace praktikum:

```
mkdir -p ~/praktikum-os/week07-bash/{bin,backup,logs,
    sampel,ruang-nama}
cd ~/praktikum-os/week04-bash
pwd
```

Kode 1.3: Membuat workspace Bash

4. Buat beberapa file contoh yang akan dipakai pada praktikum berikutnya:

```
touch sample-app.conf
touch logs/app-01.log logs/app-02.log logs/app-03.log
touch sampel/catatan-a.txt sampel/catatan-b.txt
touch sampel/backup-01.tar sampel/backup-02.tar
touch sampel/laporan-harian.log sampel/laporan-
    mingguan.log sampel/laporan-bulanan.log
touch "ruang-nama/laporan server april.txt"
touch "ruang-nama/backup [mingguan] server.conf"
ls -R
```

Kode 1.4: Membuat file contoh untuk praktikum lanjutan

Praktikum 6.2 — Membuat Ringkasan Sesi Terminal

Tujuan: membiasakan administrator memverifikasi konteks kerja sebelum melakukan maintenance.

1. Masuk ke workspace praktikum:

```
cd ~/praktikum-os/week04-bash
```

Kode 1.5: Masuk ke workspace praktikum

2. Simpan informasi sesi terminal ke file laporan:

```
{
    echo "=== RINGKASAN SESI BASH ==="
    date
    echo "User      : $(whoami)"
    echo "Hostname   : $(hostname)"
    echo "Shell login: $SHELL"
    echo "Shell aktif: $0"
```

1.2 Konfigurasi Bash (.bashrc dan .bash_profile)

```
echo "PID shell : $$"  
echo "Direktori : $(pwd)"  
} | tee session-info.txt
```

Kode 1.6: Membuat ringkasan sesi terminal

3. Verifikasi isi file laporan:

```
cat session-info.txt
```

Kode 1.7: Membaca ringkasan sesi

Catatan

Kebiasaan kecil seperti mencatat user, host, shell, dan direktori aktif sering membantu mencegah kesalahan fatal, misalnya salah bekerja di server atau direktori yang bukan target maintenance.

1.2 Konfigurasi Bash (.bashrc dan .bash_profile)

Bash memiliki beberapa file konfigurasi yang dibaca dalam konteks berbeda. Secara sederhana, `.bashrc` biasa dipakai untuk pengaturan shell interaktif: alias, prompt, fungsi, dan variabel yang ingin tersedia saat membuka terminal. Sementara itu, `.bash_profile` umumnya dipakai untuk shell login.

Pada banyak sistem Ubuntu, file login yang aktif sering kali adalah `.profile`. Namun, jika `.bash_profile` ada, maka file ini biasanya diprioritaskan oleh Bash. Karena itu, praktik aman yang umum dilakukan adalah membuat `.bash_profile` agar memanggil `.bashrc`, sehingga konfigurasi interaktif tetap konsisten.

Fungsi Perintah yang Digunakan

- `ls -la ~`: melihat file konfigurasi pada home directory.
- `cp`: membuat backup file konfigurasi.
- `cat «'EOF' » file`: menambahkan blok konfigurasi secara aman.
- `source ~/.bashrc`: menerapkan konfigurasi tanpa logout.
- `bash -l`: menjalankan shell login baru untuk pengujian.

Catatan

Jangan mengganti seluruh isi .bashrc atau .bash_profile di server produksi tanpa backup. Untuk praktikum, biasakan menambahkan blok konfigurasi sendiri dan menandainya dengan komentar yang jelas.

Praktikum 6.3 — Menambahkan Konfigurasi Aman pada .bashrc

Tujuan: memahami peran .bashrc dan menerapkan perubahan secara aman.

1. Lihat file konfigurasi Bash pada home directory:

```
ls -la ~ | grep -E 'bashrc|bash_profile|profile'
```

Kode 1.8: Melihat file konfigurasi Bash

2. Buat backup .bashrc:

```
cp ~/.bashrc ~/.bashrc.bak-praktikum
```

Kode 1.9: Backup .bashrc

3. Tambahkan blok konfigurasi praktikum:

```
cat <<'EOF' >> ~/.bashrc

# --- Praktikum Bash Shell ---
export PRAKTIKUM_BASH_DIR="$HOME/praktikum-os/week04-
    bash"
export EDITOR=nano
# --- End Praktikum Bash Shell ---

EOF
```

Kode 1.10: Menambahkan konfigurasi ke .bashrc

4. Terapkan konfigurasi tanpa logout:

```
source ~/.bashrc
echo "$PRAKTIKUM_BASH_DIR"
echo "$EDITOR"
```

Kode 1.11: Memuat ulang .bashrc

Praktikum 6.4 — Menyiapkan .bash_profile untuk Shell Login

Tujuan: memahami hubungan .bash_profile dan .bashrc pada skenario login shell.

1. Backup .bash_profile jika sudah ada:

```
[ -f ~/.bash_profile ] && cp ~/.bash_profile ~/.bash_profile.bak-praktikum
```

Kode 1.12: Backup .bash_profile jika tersedia

2. Tambahkan konfigurasi login shell:

```
cat <<'EOF' >> ~/.bash_profile

# --- Praktikum Bash Login Shell ---
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi

echo "Login Bash pada $(date '+%F %T')" >> "$HOME/praktikum-os/week07-bash/login-audit.log"
# --- End Praktikum Bash Login Shell ---

EOF
```

Kode 1.13: Menambahkan konfigurasi ke .bash_profile

3. Uji dengan membuka login shell baru:

```
bash -l
tail -n 3 ~/praktikum-os/week07-bash/login-audit.log
exit
```

Kode 1.14: Menguji .bash_profile dengan login shell

1.3 Variabel Lingkungan dan PATH

Variabel shell adalah pasangan nama dan nilai yang disimpan di dalam shell. Sebagian variabel hanya berlaku lokal pada shell saat ini, sedangkan variabel lingkungan

(*environment variable*) diekspor sehingga ikut diwariskan ke proses anak. Contoh yang sangat penting bagi administrator adalah PATH, yaitu daftar direktori yang dicari Bash saat Anda mengetik nama perintah tanpa path lengkap.

Memahami PATH sangat penting. Jika sebuah script berada di direktori yang belum masuk PATH, maka script itu tidak bisa dipanggil langsung. Sebaliknya, konfigurasi PATH yang buruk juga berbahaya, misalnya menaruh direktori yang tidak terpercaya di depan path sistem.

Fungsi Perintah yang Digunakan

- `echo $NAMA_VAR`: melihat isi variabel.
- `VAR=nilai`: membuat variabel shell lokal.
- `export VAR=nilai`: membuat variabel lingkungan.
- `printenv`: melihat environment variable.
- `which` dan `type`: melihat lokasi/jenis perintah.
- `chmod +x`: memberi izin eksekusi pada script.

Tips

Ingat perbedaannya:

- `VAR=nilai` hanya berlaku di shell saat ini.
- `export VAR=nilai` mewariskan variabel itu ke proses anak.

Praktikum 6.5 — Membedakan Variabel Shell dan Environment Variable

Tujuan: memahami perbedaan variabel lokal dan variabel lingkungan.

1. Buat variabel lokal:

```
KELAS_OS="Sistem Operasi A"
echo "$KELAS_OS"
```

Kode 1.15: Membuat variabel shell lokal

2. Buka subshell dan cek apakah variabel masih ada:

```
bash
echo "$KELAS_OS"
exit
```

Kode 1.16: Menguji variabel lokal pada subshell

3. Sekarang ubah menjadi environment variable:

```
export KELAS_OS="Sistem Operasi A"
bash
echo "$KELAS_OS"
exit
```

Kode 1.17: Membuat environment variable

4. Lihat isi PATH dan lokasi beberapa perintah:

```
echo "$PATH"
which bash
type ls
```

Kode 1.18: Melihat PATH dan lokasi perintah

Praktikum 6.6 — Menambahkan Direktori Script Pribadi ke PATH

Tujuan: membuat perintah administrasi sederhana yang bisa dipanggil dari mana saja.

1. Pastikan direktori `bin` praktikum tersedia:

```
mkdir -p ~/praktikum-os/week07-bash/bin
```

Kode 1.19: Menyiapkan direktori bin

2. Tambahkan direktori tersebut ke PATH melalui `.bashrc`:

```
cat <<'EOF' >> ~/.bashrc

# --- Praktikum PATH ---
export PATH="$HOME/praktikum-os/week07-bash/bin:$PATH"
```

```
# --- End Praktikum PATH ---

EOF

source ~/.bashrc
echo "$PATH"
```

Kode 1.20: Menambahkan direktori bin ke PATH

3. Buat script ringkasan sistem:

```
cat <<'EOF' > ~/praktikum-os/week07-bash/bin/ringkasan-
sistem
#!/usr/bin/env bash
echo "Hostname : $(hostname)"
echo "User      : $(whoami)"
echo "Uptime    : $(uptime -p)"
echo "Disk /     : "
df -h /
EOF

chmod +x ~/praktikum-os/week07-bash/bin/ringkasan-sistem
```

Kode 1.21: Membuat script ringkas-sistem

4. Jalankan script dari direktori yang berbeda:

```
cd ~
ringkas-sistem
```

Kode 1.22: Menjalankan script dari sembarang lokasi

Catatan

Menaruh direktori personal di awal PATH memudahkan pemanggilan script, tetapi tetap harus hati-hati agar nama script Anda tidak bentrok dengan perintah sistem seperti `ls`, `cp`, atau `find`.

1.4 Membuat Alias dan Fungsi Shell

Saat bekerja di terminal, sering ada perintah yang berulang-ulang dipakai. Alias membantu mempersingkat perintah yang sederhana, sedangkan fungsi shell digu-

nakan jika logika yang dibutuhkan lebih kompleks, misalnya menerima argumen, menjalankan beberapa langkah, atau memuat validasi.

Dalam pekerjaan administrator, alias cocok untuk shortcut yang aman dan sering dipakai, misalnya `ll`, `hist10`, atau `grep -color=auto`. Untuk tugas yang lebih “hidup”, seperti backup file konfigurasi dengan timestamp, fungsi shell lebih tepat digunakan daripada alias.

Fungsi Perintah yang Digunakan

- `alias`: membuat dan melihat alias.
- `unalias`: menghapus alias.
- `function_name() { ... }`: membuat fungsi shell.
- `type`: membedakan apakah sesuatu adalah alias, fungsi, builtin, atau file.
- `source ~/.bashrc`: memuat ulang alias dan fungsi.

Praktikum 6.7 — Membuat Alias Produktivitas Dasar

Tujuan: membuat shortcut perintah yang sering dipakai.

1. Tambahkan alias ke `.bashrc`:

```
cat <<'EOF' >> ~/.bashrc

# --- Praktikum Alias ---
alias ll='ls -lah --color=auto'
alias hist10='history | tail -n 10'
alias cdbashlab='cd $HOME/praktikum-os/week04-bash'
# --- End Praktikum Alias ---

EOF

source ~/.bashrc
```

Kode 1.23: Menambahkan alias ke `.bashrc`

2. Uji alias:

```
ll
hist10
```

```

cdbashlab
pwd
type ll

```

Kode 1.24: Menguji alias

Praktikum 6.8 — Membuat Fungsi Backup Konfigurasi

Tujuan: membuat fungsi shell yang dapat dipakai berulang untuk backup file konfigurasi.

1. Siapkan file konfigurasi contoh:

```

echo "PORT=8080" > ~/praktikum-os/week07-bash/sample-
  app.conf
cat ~/praktikum-os/week07-bash/sample-app.conf

```

Kode 1.25: Membuat file konfigurasi contoh

2. Tambahkan fungsi ke `.bashrc`:

```

cat <<'EOF' >> ~/.bashrc

# --- Praktikum Fungsi Shell ---
backup_conf() {
    if [ $# -ne 1 ]; then
        echo "Usage: backup_conf <file>"
        return 1
    fi

    local src="$1"
    local dst="$HOME/praktikum-os/week07-bash/backup"

    if [ ! -f "$src" ]; then
        echo "File tidak ditemukan: $src"
        return 2
    fi

    mkdir -p "$dst"
    cp -- "$src" "$dst/${basename "$src"}.${date +%F-%H%M%S}.bak"
    echo "Backup selesai di $dst"
}

```

```
}  
# --- End Praktikum Fungsi Shell ---  
  
EOF  
  
source ~/.bashrc
```

Kode 1.26: Menambahkan fungsi backup_conf ke .bashrc

3. Uji fungsi:

```
backup_conf ~/praktikum-os/week07-bash/sample-app.conf  
ls -lah ~/praktikum-os/week07-bash/backup  
type backup_conf
```

Kode 1.27: Menguji fungsi backup_conf

1.5 Penyelesaian (Completion) dan History Bash

Bash membantu pengguna mengetik lebih cepat melalui fitur *completion*. Saat Anda menekan tombol Tab, Bash mencoba melengkapi nama file, direktori, atau perintah yang sedang diketik. Fitur ini terlihat sederhana, tetapi dalam praktik administrasi sistem sangat membantu karena dapat mengurangi kesalahan pengetikan path atau nama command.

Selain itu, Bash juga menyimpan *history*, yaitu daftar perintah yang pernah dijalankan. History berguna untuk mengulang langkah kerja, menelusuri perintah yang pernah dipakai saat troubleshooting, dan mendokumentasikan workflow kerja. Fitur pencarian history dengan **Ctrl+r** adalah salah satu senjata penting administrator yang sering bekerja di terminal.

Fungsi Perintah yang Digunakan

- tombol Tab: completion nama file/perintah.
- history: melihat riwayat perintah.
- history | grep pola: mencari perintah tertentu dari history.
- !<nomor>: menjalankan ulang perintah berdasarkan nomor history.

- `!!`: menjalankan ulang perintah terakhir.
- `Ctrl+r`: pencarian interaktif ke history.

Catatan

History sangat membantu, tetapi juga berbahaya jika dipakai tanpa berpikir. Menjalankan ulang perintah dengan `!!` atau `!<nomor>` dapat berisiko jika perintah sebelumnya bersifat destruktif.

Praktikum 6.9 — Menggunakan Completion Dasar dan Melihat History

Tujuan: menggunakan fitur Tab dan history untuk mempercepat kerja.

1. Pastikan file contoh tersedia:

```
cd ~/praktikum-os/week07-bash/sampel
touch laporan-harian.log laporan-mingguan.log laporan-
      bulanan.log
ls
```

Kode 1.28: Menyiapkan file untuk completion

2. Uji completion file:
 - a) Ketik `cat lap` lalu tekan Tab dua kali.
 - b) Amati daftar file yang memiliki prefix `lap`.
 - c) Ketik lebih spesifik, misalnya `cat laporan-h` lalu tekan Tab.
3. Jalankan beberapa perintah sederhana:

```
pwd
ls -lah
date
whoami
history | tail -n 10
```

Kode 1.29: Menjalankan beberapa perintah untuk history

Praktikum 6.10 — Menelusuri Perintah Diagnostik dengan History

Tujuan: menggunakan history untuk mengulang langkah diagnosis sistem secara cepat.

1. Jalankan beberapa perintah diagnostik:

```
df -h
free -h
uptime
ps aux | head
```

Kode 1.30: Menjalankan perintah diagnostik

2. Cari ulang perintah diagnostik dari history:

```
history | grep -E 'df -h|free -h|uptime|ps aux'
```

Kode 1.31: Mencari perintah dari history

3. Jalankan ulang salah satu perintah berdasarkan nomor history:

```
!<NOMOR_HISTORY_ANDA>
```

Kode 1.32: Menjalankan ulang perintah dari history

4. Simpan potongan history ke file dokumentasi:

```
history | tail -n 20 > ~/praktikum-os/week07-bash/diag-
-history.txt
cat ~/praktikum-os/week07-bash/diag-history.txt
```

Kode 1.33: Menyimpan history ke file

Tips

Jika paket **bash-completion** tersedia pada sistem Anda, maka completion untuk banyak command akan menjadi lebih kaya. Namun, completion dasar untuk nama file, direktori, dan command sudah tersedia tanpa konfigurasi tambahan.

1.6 Wildcards dan Ekspansi Nama File

Bash mendukung *wildcards* atau *globbing* untuk mencocokkan banyak nama file sekaligus. Fitur ini sangat penting untuk pekerjaan operasional karena administrator sering harus memproses sekelompok file log, file backup, atau file konfigurasi yang namanya berpola serupa.

Selain wildcard, Bash juga memiliki ekspansi lain seperti brace expansion dan tilde expansion. Memahami perbedaan di antara semuanya membantu Anda bekerja lebih cepat, tetapi juga lebih aman, terutama ketika suatu pola akan digunakan bersama perintah seperti `cp`, `mv`, atau `rm`.

Fungsi Perintah yang Digunakan

- `*`: mencocokkan nol atau lebih karakter.
- `?`: mencocokkan tepat satu karakter.
- `[abc]` atau `[0-9]`: mencocokkan karakter tertentu/rentang.
- `{a,b,c}`: brace expansion.
- `~`: mengembangkan menjadi home directory pengguna.
- `echo pola`: mem-preview hasil ekspansi sebelum dipakai pada command berisiko.

Catatan

Kebiasaan aman saat bekerja dengan wildcard adalah mem-preview dulu hasilnya menggunakan `echo`. Jangan langsung memakai `rm *.log` atau `mv *.conf ...` tanpa melihat apa saja yang akan cocok dengan pola tersebut.

Praktikum 6.11 — Mencoba Wildcard Dasar

Tujuan: memahami pola wildcard dan ekspansi nama file.

1. Masuk ke direktori sampel:

```
cd ~/praktikum-os/week07-bash/sampel
ls
```

Kode 1.34: Masuk ke direktori sampel

1.6 Wildcards dan Ekspansi Nama File

2. Coba beberapa pola wildcard:

```
ls *.log
ls catatan-?.txt
ls backup-0[12].tar
```

Kode 1.35: Mencoba wildcard dasar

3. Coba beberapa ekspansi lain:

```
echo log-{pagi,siang,malam}.txt
echo ~
echo ~/praktikum-os/week04-bash
```

Kode 1.36: Mencoba brace dan tilde expansion

Praktikum 6.12 — Mengarsipkan Banyak Log Sekaligus

Tujuan: menggunakan wildcard secara aman untuk pekerjaan rutin administrator.

1. Siapkan file log tambahan:

```
cd ~/praktikum-os/week07-bash/logs
touch access-01.log access-02.log access-03.log
ls
```

Kode 1.37: Menyiapkan file log tambahan

2. Preview file yang akan diproses:

```
echo *.log
echo access-0?.log
```

Kode 1.38: Preview hasil wildcard

3. Pindahkan semua file log ke folder arsip:

```
mkdir -p arsip-log
mv *.log arsip-log/
ls arsip-log
```

Kode 1.39: Memindahkan file log ke folder arsip

4. Kompres folder arsip:

```
tar -czf arsip-log-$(date +%F).tar.gz arsip-log
ls -lah
```

Kode 1.40: Membuat arsip terkompresi

1.7 Quoting dan Escaping di Bash

Salah satu sumber kesalahan yang paling sering terjadi di Bash adalah masalah *quoting*. Banyak pemula menganggap shell hanya membaca kata per kata, padahal Bash melakukan banyak jenis ekspansi sebelum perintah dijalankan: ekspansi variabel, wildcard, command substitution, dan pemisahan berdasarkan spasi. Karena itu, penulisan tanda kutip yang tepat sangat penting.

Secara umum:

- **Single quote** (`'...'`) menjaga isi tetap literal.
- **Double quote** (`"..."`) tetap mengizinkan ekspansi variabel dan command substitution, tetapi mempertahankan spasi sebagai satu argumen.
- **Backslash** (`\`) digunakan untuk meng-escape satu karakter tertentu.

Dalam pekerjaan administrator, quoting yang benar sangat penting saat menangani nama file yang mengandung spasi, tanda kurung siku, tanda bintang, atau saat membangun command berbasis variabel.

Fungsi Perintah yang Digunakan

- `echo '...'`: menampilkan teks literal.
- `echo "..."`: menampilkan teks dengan ekspansi variabel.
- `\`: meng-escape karakter khusus.
- `cp - "sumber" "tujuan"`: menyalin file dengan nama yang kompleks.
- `printf '%s\n' "$var"`: menampilkan nilai variabel dengan aman.

Tips

Aturan praktis yang sangat berguna: **selalu beri tanda kutip ganda pada variabel yang merepresentasikan path atau nama file**, misalnya `"$file"`.

Praktikum 6.13 — Membedakan Single Quote, Double Quote, dan Escape

Tujuan: memahami efek tanda kutip terhadap ekspansi Bash.

1. Uji single quote dan double quote:

```
echo '$USER bekerja di $HOME'
echo "$USER bekerja di $HOME"
```

Kode 1.41: Menguji perbedaan jenis quote

2. Uji escape karakter spasi:

```
cd ~/praktikum-os/week07-bash/ruang-nama
ls laporan\ server\ april.txt
```

Kode 1.42: Menggunakan escape pada path

3. Uji akses file yang sama dengan double quote:

```
cat "laporan server april.txt"
```

Kode 1.43: Menggunakan double quote pada path

Praktikum 6.14 — Menangani File dengan Nama Sulit Secara Aman

Tujuan: memproses file yang mengandung spasi dan karakter khusus tanpa error.

1. Pastikan file target tersedia:

```
cd ~/praktikum-os/week07-bash/ruang-nama
ls -lah
```

Kode 1.44: Memastikan file target tersedia

2. Salin file dengan nama kompleks ke folder backup:

```
cp -- "backup [mingguan] server.conf" \
"$HOME/praktikum-os/week07-bash/backup/backup-
mingguan-server.conf"
```

Kode 1.45: Menyalin file dengan nama kompleks secara aman

3. Gunakan variabel untuk memproses path dengan aman:

```
file_asli="$HOME/praktikum-os/week07-bash/ruang-nama/
    backup [mingguan] server.conf"
file_salinan="$HOME/praktikum-os/week07-bash/backup/
    backup-mingguan-server-v2.conf"

cp -- "$file_asli" "$file_salinan"
ls -lah "$HOME/praktikum-os/week07-bash/backup"
```

Kode 1.46: Menggunakan variabel berisi path dengan aman

4. Tampilkan daftar file hasil backup:

```
for file in "$HOME"/praktikum-os/week07-bash/backup/*;
do
    printf 'Hasil backup: %s\n' "$file"
done
```

Kode 1.47: Menampilkan file hasil backup dengan aman

Catatan

Kesalahan klasik adalah menulis `cp $file tujuan/` tanpa tanda kutip. Jika isi variabel `$file` mengandung spasi, Bash akan memecahnya menjadi beberapa argumen dan command bisa gagal atau bahkan memproses file yang salah.

1.8 Tugas Praktikum

Peringatan

Seluruh tugas dikerjakan pada workspace praktikum sendiri, misalnya `$HOME/praktikum-os/week04-bash`, kecuali jika instruksi secara eksplisit meminta membaca file sistem seperti `/etc` atau `/proc`. Hindari mengubah file sistem penting secara langsung tanpa backup.

Tugas Praktikum 1 — Toolkit Bash Administrator Pribadi

Konteks riil: seorang administrator sering mengulang perintah yang sama setiap hari. Agar pekerjaan lebih efisien dan konsisten, ia perlu memiliki *toolkit* Bash

pribadi yang otomatis aktif setiap login.

Instruksi tugas:

1. Tambahkan konfigurasi pada `.bashrc` untuk:
 - menambahkan direktori `bin` pribadi ke `PATH`,
 - membuat minimal 2 alias yang membantu kerja harian,
 - membuat minimal 1 fungsi shell yang berguna untuk administrasi.
2. Pastikan konfigurasi tersebut aktif kembali saat membuka shell login.
3. Buat satu script sederhana di direktori `bin` pribadi, misalnya script untuk menampilkan ringkasan sistem.
4. Uji dari direktori yang berbeda untuk memastikan script dapat dipanggil tanpa menuliskan path lengkap.
5. Simpan bukti pengujian ke file `toolkit-bash-report.txt`.

Minimal luaran:

- isi blok konfigurasi yang ditambahkan ke `.bashrc`,
- output `echo $PATH`,
- output `type` untuk alias, fungsi, dan script,
- file laporan `toolkit-bash-report.txt`.

Tugas Praktikum 2 — Audit File Konfigurasi dan Logging Aman

Konteks riil: saat troubleshooting, administrator sering perlu menginventarisasi file konfigurasi dan memisahkan output normal dari pesan error.

Instruksi tugas:

1. Buat file laporan bernama `audit-konfigurasi-$(date +%F).txt`.
2. Cari file `*.conf` di dalam `/etc` dan simpan hasilnya ke file laporan.
3. Catat jumlah total file konfigurasi yang ditemukan.
4. Jika ada pesan error, simpan ke file terpisah, misalnya `audit-error.log`.
5. Tampilkan isi laporan ke terminal dan sekaligus simpan menggunakan `tee`.
6. Tambahkan ringkasan singkat 3–5 baris yang menjelaskan mengapa pemisahan `stdout` dan `stderr` penting dalam audit sistem.

Syarat konsep yang harus muncul:

- redirection `>`, `2>`, atau `&>`,
- pipeline,
- tee,
- penggunaan variabel atau `command substitution`.

Minimal luaran:

- file laporan audit,
- file log error,
- perintah yang digunakan,
- analisis singkat hasil audit.

Tugas Praktikum 3 — Mini Health Check Harian Server

Konteks riil: administrator perlu membuat pemeriksaan cepat (*health check*) untuk mengetahui kondisi dasar server sebelum dan sesudah maintenance.

Instruksi tugas:

1. Buat script Bash bernama `daily-healthcheck` pada direktori `bin` pribadi.
2. Script minimal harus menampilkan:
 - tanggal dan waktu,
 - hostname,
 - user aktif,
 - shell aktif,
 - uptime,
 - penggunaan memori,
 - penggunaan filesystem root,
 - 10 baris terakhir history command yang relevan dengan pengecekan.
3. Simpan hasil ke file log harian, misalnya `healthcheck-$(date +%F).log`.
4. Tampilkan hasil ke terminal dan ke file secara bersamaan.
5. Jika Anda menggunakan pipeline dengan `tee`, cek juga status exit command

utama.

Syarat konsep yang harus muncul:

- environment variable,
- PATH,
- alias atau fungsi pendukung,
- history,
- tee,
- penanganan error dasar.

Minimal luaran:

- file script yang executable,
- contoh isi file log hasil eksekusi,
- penjelasan singkat fungsi tiap bagian script.

Tugas Praktikum 4 — Penanganan File dengan Nama Kompleks dan Arsip Aman

Konteks riil: file hasil backup, ekspor, atau laporan sering memiliki nama yang mengandung spasi atau karakter khusus. Administrator harus tetap dapat memproses file-file tersebut tanpa salah target.

Instruksi tugas:

1. Buat minimal 4 file contoh dengan nama yang bervariasi, termasuk:
 - nama file yang mengandung spasi,
 - nama file yang mengandung tanda kurung siku atau karakter khusus,
 - file dengan pola nama serupa untuk diuji dengan wildcard.
2. Tunjukkan perbedaan hasil jika file diakses tanpa quoting dan dengan quoting yang benar.
3. Lakukan *preview* wildcard dengan **echo** sebelum dipakai untuk operasi nyata.
4. Salin file-file tersebut ke direktori backup dengan nama yang aman.
5. Buat arsip **tar.gz** dari hasil backup.
6. Simpan riwayat perintah yang Anda gunakan ke file **riwayat-arsip.txt**.

Syarat konsep yang harus muncul:

- single quote, double quote, dan escaping,
- wildcard,
- variabel path,
- history,
- operasi file lanjutan yang aman.

Minimal luaran:

- daftar file awal,
- daftar file hasil backup,
- file arsip `tar.gz`,
- file `riwayat-arsip.txt`,
- refleksi singkat tentang pentingnya quoting di Bash.

1.9 Rangkuman

- Bash adalah shell yang sangat umum digunakan pada Linux dan menjadi alat utama interaksi administrator dengan sistem.
- `.bashrc` dipakai untuk konfigurasi shell interaktif, sedangkan `.bash_profile` dipakai pada konteks login shell.
- Variabel lingkungan dan `PATH` menentukan konteks kerja shell serta bagaimana Bash menemukan command dan script.
- Alias cocok untuk shortcut sederhana, sedangkan fungsi shell cocok untuk logika yang lebih kompleks dan berulang.
- Completion dan history membantu mempercepat kerja serta mengurangi kesalahan pengetikan.
- Wildcard dan ekspansi nama file memudahkan pemrosesan banyak file sekaligus, tetapi harus digunakan dengan kebiasaan preview yang aman.
- Quoting dan escaping adalah fondasi penting agar Bash menangani variabel, spasi, dan karakter khusus dengan benar.

1.10 Referensi dan Bahan Belajar Lanjutan

Untuk pendalaman materi, mahasiswa disarankan mempelajari referensi berikut:

1.10 Referensi dan Bahan Belajar Lanjutan

1. GNU Bash Reference Manual.
2. Bash manual page (`man bash`).
3. Bash Guide for Beginners.
4. William Shotts, *The Linux Command Line*.
5. Evi Nemeth, Garth Snyder, Trent R. Hein, Ben Whaley, Dan Mackin, *UNIX and Linux System Administration Handbook*, 5th Edition.
6. Arnold Robbins dan Nelson H. F. Beebe, *Classic Shell Scripting*.
7. Mark G. Sobell, *A Practical Guide to Linux Commands, Editors, and Shell Programming*.
8. Richard Blum dan Christine Bresnahan, *Linux Command Line and Shell Scripting Bible*.